PARTIE 3 · REACT

Getting Started with React

Introduction progressive à React après JavaScript moderne et les notions Frontend, Backend et technologies Web : SPA, rendu React, JSX, Vite, composants, props et listes.

SÉANCE 4 · SÉANCE 5

React
Construire des interfaces utilisateur dynamiques et réactives.

Components
Découper l'interface en blocs de code autonomes et réutilisables.

SPA
Naviguer et changer de vue sans jamais recharger toute la page.

🎯 OBJECTIF GLOBAL

Être capable de créer un projet React moderne avec Vite, de comprendre le rendu et la syntaxe JSX, de styliser l'interface, puis de construire une première application structurée avec des composants, des props et des listes.

À la fin de ce parcours, vous saurez :

1. Expliquer le rôle de React.
2. Comprendre une SPA.
3. Expliquer le Virtual DOM.
4. Écrire et styliser du JSX.
5. Comprendre Babel, le bundling et Vite.
6. Créer, composer et réutiliser des composants avec des props.

⚠ Pré-requis

Maîtrise de JavaScript ES6 : Fonctions, objets, tableaux, déstructuration, spread operator, modules (import/export), fonctions fléchées et méthodes de tableau (spécifiquement `map()`).

TRANSITION JS → REACT

Pourquoi utiliser React ?

React devient particulièrement utile lorsque l'interface contient plusieurs zones dynamiques qui doivent rester cohérentes avec les données de l'application.

Le problème en JS classique

1. L'approche est "Impérative".
2. On sélectionne le DOM manuellement (`querySelector()`).
3. On modifie l'affichage étape par étape.
4. Conséquence : la logique d'interface peut devenir difficile à maintenir à mesure que l'application grandit.

La solution avec React

1. L'approche est "Déclarative".
2. On décrit l'interface finale désirée.
3. Les données changent, React s'occupe de la mise à jour du DOM.
4. Conséquence : Le code reste organisé et prévisible.

Comparaison par le code

JAVASCRIPT + DOM MANIPULATION

script.js — CODE

```
const titre = document.querySelector("#titre");

// Modification manuelle étape par étape
titre.textContent = "Liste des Étudiants";
titre.classList.add("text-indigo-600");
```

APPROCHE DÉCLARATIVE REACT

Titre.jsx — CODE

```
function Titre({ texte }) {
  return (
    <h1 className="text-indigo-600">
      {texte}
    </h1>
  );
}
```

💡 Le changement de paradigme

En JS classique, vous expliquez au navigateur **comment** modifier le DOM étape par étape. En React, vous décrivez simplement **ce que** vous voulez afficher selon l'état des données.

ARCHITECTURE

Comprendre une SPA

SPA signifie **Single Page Application**. Le navigateur charge une page HTML initiale. Dans notre application, React gère l'interface située à l'intérieur de `#root`. Les changements de vue se font ensuite côté client sans demander une nouvelle page HTML complète à chaque navigation.

Site Classique (Multi-pages)

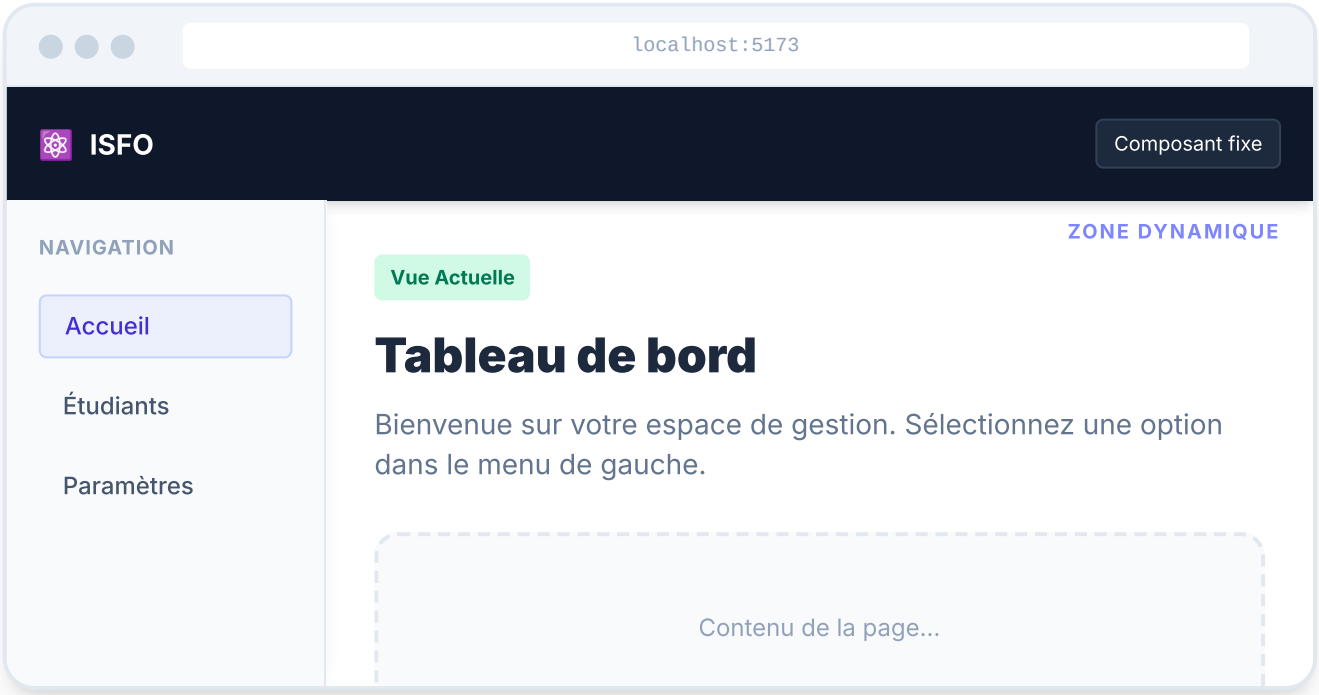
Dans une navigation multi-pages classique, un changement de page peut provoquer une nouvelle requête de document HTML au serveur.

Application SPA (React)

Dans une SPA, les zones communes peuvent rester affichées tandis que la zone principale change dynamiquement côté client.

Démonstration Interactive

🔄 Simuler un clic de menu



RENDU REACT

Du rendu au DOM réel

Le terme **Virtual DOM** décrit la représentation en mémoire de l'interface. Pour comprendre React correctement, retenons surtout le cycle : **Trigger → Render → Reconciliation → Commit**.

⚡

1. Trigger — un rendu est demandé

Au démarrage de l'application ou après une mise à jour d'état dans un composant.

🧩

2. Render — React appelle les composants

React calcule ce que l'interface devrait afficher à partir des props et de l'état.

🔍

3. Reconciliation — déterminer les changements

React compare le nouveau résultat du rendu avec le précédent afin de déterminer les opérations nécessaires.

💻

4. Commit — mise à jour du DOM

React applique au DOM les opérations nécessaires pour que l'écran corresponde au dernier rendu.

À retenir

Un nouveau rendu ne signifie pas que tout le DOM est reconstruit. React calcule l'interface puis applique uniquement les changements nécessaires au navigateur.

SYNTAXE

Règles essentielles du JSX

JSX est une extension de syntaxe qui permet de décrire l'interface dans JavaScript. Il ressemble à HTML, mais il suit quelques règles spécifiques.

1 · Un seul élément racine

Retournez un parent unique ou un Fragment `<>...</>`.

2 · className

En JSX, utilisez `className` pour les classes CSS.

3 · Fermer les balises

`<img />`, `<input />`, `<br />`...

4 · { } pour JavaScript

Les accolades permettent d'insérer une **expression JavaScript** dans JSX.

5 · Composants avec majuscule

`<StudentCard />` désigne un composant ; `<div>` un élément HTML.

6 · JSX reste dans JavaScript

Variables, ternaires, appels de fonctions et `map()` peuvent produire l'interface.

Exemple regroupant les règles principales

ProfilEtudiant.jsx — CODE

```jsx
function ProfilEtudiant() {
  const nom = "Yassine";
  const note = 14;

  return (
    <>
      <div className="rounded-xl bg-white p-4">
        <h2>
          {nom}
        </h2>

        <p>
          Note : {note} / 20
        </p>

        <img src="/avatar.png" alt="Avatar" />
      </div>
    </>
  );
}
```

Idée clé : JSX ne remplace pas JavaScript. C'est une syntaxe utilisée à l'intérieur de JavaScript pour décrire l'interface.

JSX · STYLES

Appliquer du style en JSX

En React, nous pouvons appliquer du style avec `className`, avec la propriété `style` ou avec une expression JavaScript permettant de changer le style dynamiquement.

1. Utiliser className

En JSX, on utilise `className` à la place de `class`. C'est la méthode que nous utilisons principalement avec Tailwind CSS.

StudentCard.jsx — CODE

```jsx
function StudentCard() {
  return (
    <div className="bg-white p-4 rounded-xl shadow">
      <h3 className="text-xl font-bold text-indigo-600">
        Yassine
      </h3>
    </div>
  );
}
```

2. Utiliser style avec un objet JavaScript

La propriété `style` reçoit un **objet JavaScript**. Les propriétés CSS sont écrites en **camelCase**.

Style inline — CODE

```jsx
<h3
  style={{
    color: "blue",
    fontSize: "20px",
    fontWeight: "bold"
  }}
>
  Yassine
</h3>
```

CSS → JSX

- `font-size` → `fontSize`
- `background-color` → `backgroundColor`
- `margin-top` → `marginTop`

Les propriétés CSS contenant un tiret sont transformées en notation **camelCase**.

Pourquoi deux paires d'accolades ?

`style={ ... }` — Les accolades externes permettent d'utiliser une expression JavaScript dans JSX.

+

`{ color: "blue" }` — Les accolades internes représentent l'objet JavaScript contenant le style.

```jsx
style={{ color: "blue" }}
```

3. Le style peut être stocké dans une variable

StudentCard.jsx — CODE

```jsx
function StudentCard() {
  const monStyle = {
    color: "blue",
    fontSize: "20px",
    fontWeight: "bold"
  };

  return (
    <h3 style={monStyle}>
      Yassine
    </h3>
  );
}
```

Style dynamique

Le style peut dépendre d'une donnée.

```jsx
style={{
  color: note >= 10
    ? "green"
    : "red"
}}
```

Avec Tailwind CSS

Une expression JavaScript peut également choisir les classes.

```jsx
className={
  note >= 10
    ? "font-bold text-green-600"
    : "font-bold text-red-600"
}
```

À retenir : `className` applique des classes CSS ou Tailwind. `style` reçoit un objet JavaScript. Les styles et les classes peuvent également être calculés dynamiquement à partir des données.

OUTILLAGE

Babel : transformer le code

Avant Vite, il est utile de comprendre le concept de **transformation**. Babel est un outil capable de transformer du code JavaScript selon une configuration, notamment le JSX et certaines syntaxes modernes.

Code écrit par le développeur

source.jsx — CODE

```jsx
const titre = (
  <h1 className="text-indigo-600">
    Bonjour
  </h1>
);
```

JSX + JavaScript moderne → Transformation → JavaScript exploitable

À quoi sert Babel ?

- Appliquer des transformations au code.
- Transformer JSX selon le mode utilisé.
- Utiliser des presets et plugins.
- Aider à cibler différents environnements.

Dans notre cours

Nous devons comprendre le rôle de la transformation, mais nous n'allons pas configurer Babel manuellement. Vite prend en charge l'outillage nécessaire à notre projet de départ.

À retenir : Babel est un outil de transformation ; JSX n'est pas directement du HTML.

OUTILLAGE

Bundlers : préparer l'application

Une application moderne contient de nombreux modules JavaScript, composants, feuilles de style et ressources. Un bundler analyse leurs dépendances et prépare les fichiers nécessaires au déploiement.

Plusieurs modules et ressources

Projet React — CODE

```
src/
├── App.jsx
├── components/
│   ├── Header.jsx
│   ├── Sidebar.jsx
│   └── StudentCard.jsx
├── index.css
└── assets/
    └── logo.svg
```



Le bundler

- Analyse les `import`.
- Construit le graphe de dépendances.
- Prépare les ressources de production.
- Peut optimiser et découper le code.

Exemple connu

Webpack a longtemps été une référence majeure. Aujourd'hui, des outils comme Vite proposent une expérience de développement plus intégrée et simplifient fortement la configuration.

Message pédagogique : Vite ne supprime pas les notions de transformation et de bundling : il les rend beaucoup plus simples à utiliser dans notre workflow.

OUTILLAGE

Créer le projet avec Vite

Vite est un outil moderne de développement et de build. Pour apprendre les bases de React, nous l'utilisons pour créer rapidement un projet client avec JSX, modules, serveur de développement et build.

Création du projet

Terminal (Bash / PowerShell) CODE

```
node -v
npm -v

npm create vite@latest mon-projet-react -- --template react
cd mon-projet-react
npm install
npm run dev

# Ouvrir ensuite l'adresse locale indiquée par Vite
# Exemple : http://localhost:5173/
```

Comprendre l'arborescence générée

```
📁 mon-projet-react/
├── 📁 node_modules/
├── 📁 src/
│   ├── 📄 main.jsx
│   ├── 📄 App.jsx
│   └── 📄 index.css
├── 📄 index.html
└── 📄 package.json
```

index.html La page web vierge. Elle contient l'élément `<div id="root"></div>`.

main.jsx Le point d'entrée. Il capture la `div#root` et lui greffe l'application React complète.

App.jsx Le composant racine de votre code. C'est ici que l'on assemble les autres composants.

package.json Le "carnet de santé" de l'app. Il liste les scripts de lancement et les dépendances.

Note : les versions récentes de Vite exigent une version récente de Node.js. Si npm signale une incompatibilité, mettez Node.js à jour avant de continuer.

POINT D'ENTRÉE

DÉMARRAGE DU PROJET

De index.html à App.jsx

Le navigateur charge `index.html`, puis `main.jsx` crée la racine React dans `#root` et affiche le composant `<App />`.



Point d'entrée React

src/main.jsx — CODE

```jsx
import { StrictMode } from "react";
import { createRoot } from "react-dom/client";
import App from "./App.jsx";
import "./index.css";

createRoot(document.getElementById("root")).render(
  <StrictMode>
    <App />
  </StrictMode>
);
```

Tailwind CSS avec Vite

Installation

Terminal — CODE

```
npm install tailwindcss @tailwindcss/vite
```

Plugin Tailwind pour Vite

vite.config.js — CODE

```jsx
import { defineConfig } from "vite";
import react from "@vitejs/plugin-react";
import tailwindcss from "@tailwindcss/vite";

export default defineConfig({
  plugins: [
    react(),
    tailwindcss()
  ]
});
```

Importer Tailwind

src/index.css — CODE

```jsx
@import "tailwindcss";
```

DÉVELOPPEMENT · PRODUCTION

StrictMode, build & preview

Avant de poursuivre avec les composants, distinguons le mode de développement du build de production et comprenons le rôle de `StrictMode` présent dans `main.jsx`.

1. Le rôle de StrictMode

src/main.jsx — CODE

```jsx
import { StrictMode } from "react";

<StrictMode>
  <App />
</StrictMode>
```

En développement

- `StrictMode` aide à repérer certains problèmes potentiels.
- Il n'ajoute aucun élément visible à l'interface.
- React peut exécuter certains comportements plus d'une fois en développement pour détecter du code problématique.
- Ce comportement de vérification n'est pas appliqué de la même manière au build de production.

2. Trois commandes à distinguer

Terminal — CODE

```
npm run dev

npm run build

npm run preview
```

NPM RUN DEV

Lance le serveur de développement avec rechargement rapide pendant le codage.

NPM RUN BUILD

Prépare une version optimisée de l'application dans le dossier `dist/`.

NPM RUN PREVIEW

Permet de vérifier localement le build généré avant son déploiement.

À retenir : `dev` sert à développer, `build` prépare la version de production et `preview` permet de tester localement cette version.

ARCHITECTURE DES COMPOSANTS

Découper l'Interface

En React, on ne code pas un fichier géant. On conçoit l'interface visuelle en l'assemblant comme des pièces de Lego. Créons une architecture classique : **Header + Sidebar + Main**.

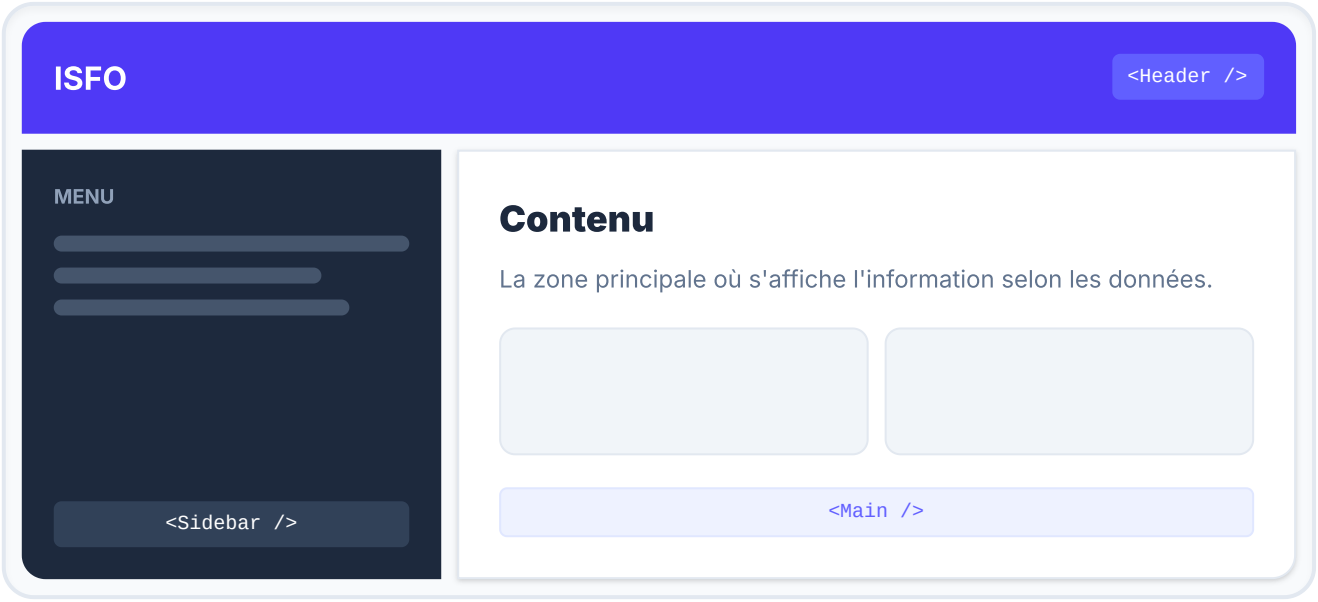
NOUVEAU BLOC PÉDAGOGIQUE

Après avoir compris l'environnement React et son outillage, nous passons maintenant à la construction réelle de l'interface avec des composants réutilisables.



Démonstration : Visualisation

🔍 Isoler un composant



💡 Conseil d'organisation

Créez un dossier `src/components/` dans votre projet pour y ranger ces fichiers séparément. Cela permet de garder votre composant central `App.jsx` propre et lisible.

PRATIQUE

Créer Header & Sidebar

Chaque composant React s'écrit comme une fonction JavaScript. **Règle stricte :** Le nom de la fonction doit toujours commencer par une **lettre majuscule** (ex: `Header`, jamais `header`).

src/components/Header.jsx — CODE

```jsx
export default function Header() {
  return (
    <header className="bg-indigo-600 p-5 text-white shadow-md">
      <h1 className="text-2xl font-bold">
        Gestion des étudiants
      </h1>
    </header>
  );
}
```

src/components/Sidebar.jsx — CODE

```jsx
export default function Sidebar() {
  return (
    <aside className="w-64 bg-slate-900 p-5 text-white min-h-screen">
      <h2 className="font-bold text-slate-400 uppercase text-sm mb-4">
        Menu
      </h2>
      <ul className="space-y-3 text-sm">
        <li className="hover:text-indigo-400 cursor-pointer">Accueil</li>
        <li className="hover:text-indigo-400 cursor-pointer">Étudiants</li>
        <li className="hover:text-indigo-400 cursor-pointer">Notes</li>
      </ul>
    </aside>
  );
}
```

⚠️ La nécessité d'exporter

L'instruction `export default` exporte le composant comme export principal du fichier afin de pouvoir l'importer dans un autre module, par exemple dans `App.jsx`.

PRATIQUE (SUITE)

Main et Composition

Créons le composant de contenu central **Main**, puis effectuons l'étape finale de **Composition** : rassembler ces trois "briques" (Header, Sidebar, Main) dans la coquille mère `App.jsx`.

src/components/Main.jsx — CODE

```jsx
export default function Main() {
  return (
    <main className="flex-1 p-8 bg-slate-50">
      <h2 className="text-3xl font-black text-slate-800">Bienvenue</h2>
      <p className="mt-3 text-slate-600">Interface principale chargée.</p>
    </main>
  );
}
```

Assemblage final : Le composant Parent

src/App.jsx — CODE

```jsx
// 1. Importation stricte des composants créés
import Header from "./components/Header.jsx";
import Sidebar from "./components/Sidebar.jsx";
import Main from "./components/Main.jsx";

export default function App() {
  return (
    <div className="flex flex-col h-screen">
      {/* 2. On utilise les composants sous forme de balises auto-fermantes */}
      <Header />

      <div className="flex flex-1">
        <Sidebar />
        <Main />
      </div>
    </div>
  );
}
```

REACT · FLUX DE DONNÉES

Comprendre les props

Les **props** permettent à un composant parent de transmettre des données à un composant enfant. Avant la déstructuration, il faut comprendre que `props` est simplement un objet JavaScript. Le flux de données va du parent vers l'enfant, et le composant enfant ne doit pas modifier les props qu'il reçoit.

Parent
`<StudentCard nom="Yassine" note={14} />`

→

Enfant
`function StudentCard(props)`

1. Le parent envoie des données

App.jsx — CODE

```jsx
function App() {
  return (
    <StudentCard nom="Yassine" note={14} />
  );
}
```

2. L'enfant reçoit un objet props

StudentCard.jsx — CODE

```jsx
function StudentCard(props) {
  return (
    <div className="rounded-xl bg-white p-4 shadow">
      <h3 className="font-bold">
        {props.nom}
      </h3>

      <p className="text-slate-500">
        Note : {props.note} / 20
      </p>
    </div>
  );
}
```

Mentalement, on peut imaginer :

Objet JavaScript — CODE

```jsx
const props = {
  nom: "Yassine",
  note: 14
};
```

À retenir : les props ne sont pas une syntaxe spéciale mystérieuse. Le composant reçoit un objet contenant les valeurs envoyées par son parent.

REACT · PROPS

Déstructurer les props

Une fois que les étudiants ont compris `props.nom` et `props.note`, on peut appliquer une notion déjà connue en JavaScript : la **déstructuration**.

1. Sans déstructuration

StudentCard.jsx — CODE

```jsx
function StudentCard(props) {
  return (
    <div>
      <h3>{props.nom}</h3>
      <p>Note : {props.note} / 20</p>
    </div>
  );
}
```

2. Déstructuration à l'intérieur de la fonction

StudentCard.jsx — CODE

```jsx
function StudentCard(props) {
  const { nom, note } = props;

  return (
    <div>
      <h3>{nom}</h3>
      <p>Note : {note} / 20</p>
    </div>
  );
}
```

3. Déstructuration directement dans les paramètres

StudentCard.jsx — CODE

```jsx
function StudentCard({ nom, note }) {
  return (
    <div>
      <h3>{nom}</h3>
      <p>Note : {note} / 20</p>
    </div>
  );
}
```

Rappel JavaScript

JavaScript — CODE

```javascript
const etudiant = {
  nom: "Yassine",
  note: 14
};

const { nom, note } = etudiant;
```

À retenir : `function StudentCard({ nom, note })` est simplement de la déstructuration JavaScript appliquée à l'objet `props`.

REACT · PROPS

Passer des valeurs simples

Les props peuvent recevoir plusieurs types de valeurs : chaînes, nombres, booléens ou expressions JavaScript.

Chaîne

`nom="Yassine"`
Une chaîne écrite directement peut utiliser les guillemets.

Nombre

`note={14}`
Une valeur JavaScript s'écrit entre accolades.

Booléen

`admis={true}`
Les booléens sont aussi des valeurs JavaScript.

Parent

App.jsx — CODE

```jsx
function App() {
  return (
    <StudentCard
      nom="Yassine"
      note={14}
      admis={true}
    />
  );
}
```

Enfant

StudentCard.jsx — CODE

```jsx
function StudentCard({ nom, note, admis }) {
  return (
    <div className="rounded-xl bg-white p-4 shadow">
      <h3 className="font-bold">
        {nom}
      </h3>

      <p>
        Note : {note} / 20
      </p>

      <p>
        {admis ? "Admis" : "Non admis"}
      </p>
    </div>
  );
}
```

"Yassine" → chaîne écrite directement

{14} → expression JavaScript

{true} → expression JavaScript

À retenir : les guillemets conviennent pour une chaîne littérale ; les accolades { } permettent de passer une expression JavaScript.

REACT · PROPS

Passer un objet

Au lieu d'envoyer chaque propriété séparément, on peut transmettre un objet complet au composant enfant.

1. Créer l'objet dans le parent

App.jsx — CODE

```jsx
const etudiant = {
  id: 1,
  nom: "Yassine",
  note: 14,
  groupe: "DDOWFS201"
};

function App() {
  return (
    <StudentCard etudiant={etudiant} />
  );
}
```

2. Recevoir l'objet dans l'enfant

StudentCard.jsx — CODE

```jsx
function StudentCard({ etudiant }) {
  return (
    <div className="rounded-xl bg-white p-4 shadow">
      <h3 className="font-bold">
        {etudiant.nom}
      </h3>

      <p>
        Note : {etudiant.note} / 20
      </p>

      <p>
        Groupe : {etudiant.groupe}
      </p>
    </div>
  );
}
```

3. Déstructurer l'objet reçu

StudentCard.jsx — CODE

```jsx
function StudentCard({ etudiant }) {
  const { nom, note, groupe } = etudiant;

  return (
    <div>
      <h3>{nom}</h3>
      <p>Note : {note} / 20</p>
      <p>Groupe : {groupe}</p>
    </div>
  );
}
```

À retenir : une prop peut contenir un objet complet. Cela évite parfois de passer beaucoup de props séparées.

REACT · PROPS & LISTES

Passer une liste simple

Une prop peut également contenir un tableau. Le composant enfant peut ensuite utiliser `map()` pour produire plusieurs éléments JSX.

Parent

App.jsx — CODE

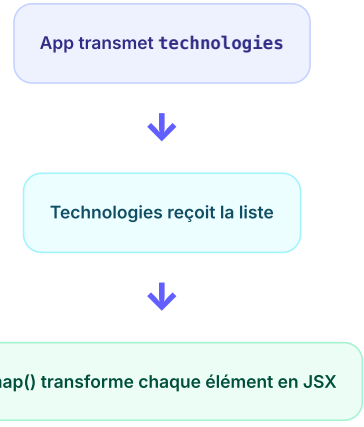
```jsx
const technologies = [
  "HTML",
  "CSS",
  "JavaScript",
  "React"
];

function App() {
  return (
    <Technologies technologies={technologies} />
  );
}
```

Enfant

Technologies.jsx — CODE

```jsx
function Technologies({ technologies }) {
  return (
    <ul className="space-y-2">
      {technologies.map((technologie) => {
        return (
          <li key={technologie}>
            {technologie}
          </li>
        );
      })}
    </ul>
  );
}
```



À retenir : une liste peut être passée comme n'importe quelle autre valeur JavaScript. Ensuite, `map()` permet de générer l'interface à partir des données.

REACT · PROPS & LISTES

Passer une liste d'objets

C'est le cas le plus proche de notre mini-application : le parent transmet une liste d'étudiants, puis un composant transforme chaque objet en `<StudentCard />`.

1. Les données

data.js ou App.jsx — CODE

```
const etudiants = [
  { id: 1, nom: "Yassine", note: 14 },
  { id: 2, nom: "Youssef", note: 16 },
  { id: 3, nom: "Ghita", note: 18 }
];
```

2. Le parent transmet la liste

App.jsx — CODE

```
function App() {
  return (
    <StudentList etudiants={etudiants} />
  );
}
```

3. StudentList utilise map()

StudentList.jsx — CODE

```
function StudentList({ etudiants }) {
  return (
    <div className="grid gap-4">
      {etudiants.map((etudiant) => {
        return (
          <StudentCard
            key={etudiant.id}
            etudiant={etudiant}
          />
        );
      })}
    </div>
  );
}
```

4. StudentCard reçoit un seul objet

StudentCard.jsx — CODE

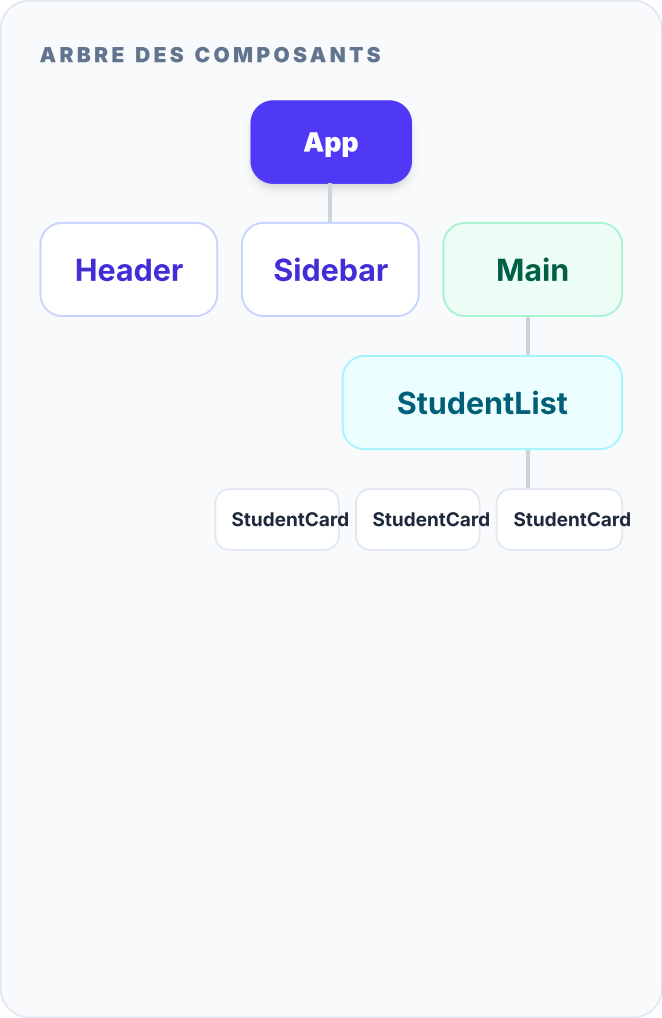
```
function StudentCard({ etudiant }) {
  return (
    <div>
      <h3>{etudiant.nom}</h3>
      <p>Note : {etudiant.note} / 20</p>
    </div>
  );
}
```

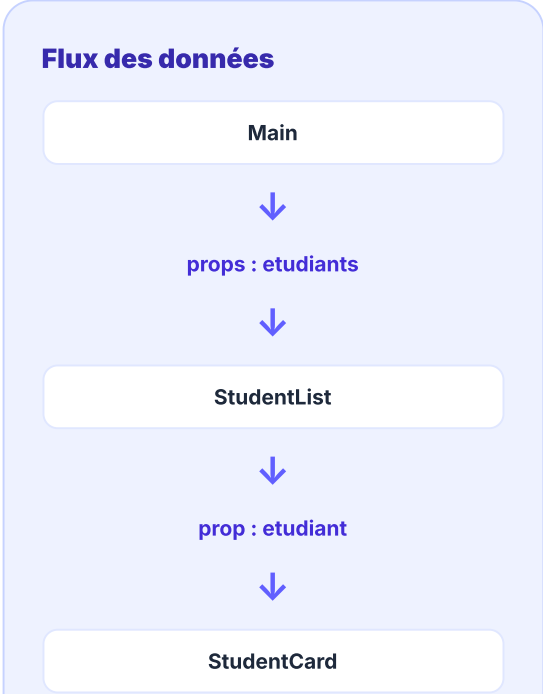
Architecture obtenue : `App → StudentList → map() → StudentCard` L'attribut spécial key aide React à identifier chaque élément de la liste de manière stable.key donne à chaque élément de la liste une identité stable.

ARCHITECTURE REACT

Visualiser l'arbre des composants

Une application React est un arbre de composants. Les composants sont imbriqués, tandis que les données sont transmises du parent vers l'enfant à l'aide des props.





Flux unidirectionnel

Dans les exemples de ce cours, les données descendent du parent vers l'enfant. L'enfant utilise les props reçues pour produire son interface.

Relier architecture et code

Main.jsx — CODE

```jsx
<StudentList etudiants={etudiants} />

// StudentList transmet ensuite chaque objet
<StudentCard
  key={etudiant.id}
  etudiant={etudiant}
/>
```

À retenir : pensez toujours à deux choses : **qui contient qui ?** pour l'arbre des composants, et **qui envoie les données à qui ?** pour le flux des props.

OUTILS DU DÉVELOPPEUR

Déboguer une application React

Lorsqu'une erreur apparaît, il faut d'abord lire le message fourni par le navigateur et localiser le composant ou le fichier concerné avant de modifier le code.

Console

Affiche les erreurs JavaScript, les avertissements et les résultats de `console.log()`.

Network

Permet de vérifier les fichiers et ressources chargés par l'application.

React DevTools

Permet d'explorer l'arbre des composants et d'observer les props reçues.

Une méthode simple en 4 étapes

1 **Lire le message d'erreur**

Ne corrigez pas au hasard : identifiez d'abord le fichier, la ligne et le type d'erreur.

2 **Vérifier la syntaxe et les imports**

Contrôlez les balises JSX, accolades, chemins de fichiers et noms de composants.

3 **Observer les données**

Utilisez temporairement `console.log()` ou React DevTools pour vérifier les props.

4 **Corriger puis retester**

Faites une correction ciblée, sauvegardez et vérifiez à nouveau le navigateur.

StudentCard.jsx — CODE

```
function StudentCard({ etudiant }) {
  console.log(etudiant);

  return (
    <h3>{etudiant.nom}</h3>
  );
}
```

Bon réflexe : comprendre le message d'erreur avant de chercher une solution.

BONNES PRATIQUES

Les erreurs React à éviter

Ces erreurs sont fréquentes au début. Les reconnaître rapidement permet de gagner du temps lors des travaux pratiques.

1. Nom en minuscule — À éviter

```
function studentCard()
```

Un composant React doit commencer par une majuscule : `StudentCard`.

2. Utiliser class — JSX

```
<div class="card">
```

En JSX, utilisez `className="card"`.

3. Balise non fermée — JSX

```
<img src="avatar.png">
```

Les balises sans contenu doivent être fermées : `<img />`.

4. Mauvais import — Modules

```
import Header from "./Header.jsx"
```

Vérifiez le chemin, le nom du fichier et le type d'export utilisé.

5. Afficher un objet directement — Données

```
<p>{etudiant}</p>
```

Affichez une propriété, par exemple `{etudiant.nom}`.

6. Oublier key dans map() — Listes

```
<StudentCard etudiant={item} />
```

Ajoutez une identité stable : `key={item.id}`.

Attention aux accolades

INCORRECT POUR UN NOMBRE

```
<StudentCard note={{14}} />
```

Ici, `{{14}}` représente un objet, pas le nombre 14.

CORRECT

```
<StudentCard note={14} />
```

Une seule paire d'accolades suffit pour passer une valeur JavaScript.

Méthode : lorsque l'application ne fonctionne pas, vérifiez dans cet ordre : syntaxe JSX → imports → props → données → key.

TRAVAIL PRATIQUE

Mini App : Gestion Scolaire

Appliquez les concepts étudiés : initialisez Vite, créez une architecture de fichiers propre, construisez le layout et utilisez les props pour afficher la liste des étudiants.

Étapes à suivre :

1. Initialiser le projet avec **Vite**.
2. Installer Tailwind CSS avec `tailwindcss` + `@tailwindcss/vite`, puis ajouter le plugin dans `vite.config.js`.
3. Créer les composants : `Header`, `Sidebar`, `Main` et `StudentCard`.
4. Assembler l'architecture dans `App.jsx`, puis afficher les étudiants avec `map()` et une `key` stable.
5. Initialiser **Git**, publier le projet dans un dépôt **GitHub** et envoyer le **lien du repository** au formateur.

Données à utiliser dans Main :

Main.jsx — CODE

```
const etudiants = [
  { id: 1, nom: "Yassine", note: 14 },
  { id: 2, nom: "Youssef", note: 16 },
  { id: 3, nom: "Abderrahmane", note: 17 },
  { id: 4, nom: "Ghita", note: 18 },
  { id: 5, nom: "Sara", note: 9 }
];

// Utiliser map() pour générer les cartes
// key={etudiant.id} pour identifier chaque élément
```

✓ REMISE OBLIGATOIRE

Le travail doit être publié sur **GitHub**. Envoyez au formateur le **lien du repository** contenant votre projet.

EXEMPLE

`https://github.com/votre-compte/gestion-scolaire-react`

🚀 BONUS · OPTIONNEL

Dans `StudentCard.jsx`, affichez la note en **vert** si elle est ≥ 10 et en **rouge** sinon.

Indice : `note >= 10 ? "text-green-600" : "text-red-600"`

BILAN

Synthèse & Évaluation

React & SPA

React gère une interface par composants ; dans notre SPA, les vues évoluent côté client dans la racine React.

Render & Commit

React calcule le rendu puis applique au DOM les opérations nécessaires.

JSX

Une syntaxe dans JavaScript pour décrire l'interface avec éléments, composants et expressions.

Babel · Bundling · Vite

Transformation, bundling, Vite et commandes dev/build/preview structurent le workflow du projet.

Components · Props · Data Flow

Les composants forment un arbre ; les données descendent du parent vers l'enfant via les props.

map() & key

Les tableaux de données peuvent produire des listes de composants avec une identité stable.

🛠️ **Débogage :** Console, Network et React DevTools permettent de comprendre plus rapidement les erreurs et le flux des props.

Questions pour valider l'acquis

1. **Que signifient les étapes Trigger, Render et Commit dans React ?**
2. **Citez quatre règles importantes du JSX.**
3. **Quelle différence faites-vous entre transformation du code, bundling et Vite ?**
4. **Pourquoi utilise-t-on key lorsqu'un tableau est rendu avec `map()` ?**

PROCHAINE ÉTAPE

Interactivité : Events & useState

Maintenant que nous savons dessiner l'interface et recevoir des propriétés, nous apprendrons à réagir aux clics utilisateurs (Events) et à faire évoluer la mémoire interne du composant avec le hook **useState** !